Session 23 Homework — Solution Walkthrough

Status: PREPARED DRAFT. Published with the official solution, for independent study.

What the homework asked you to prove

A real Issue Detail page at a real third nested dynamic route, rendering one real issue's own real fields — then a real, disclosed bug (found by actually testing a non-numeric id against the real backend) fixed on all three real dynamic routes this session built.

What the official solution changes

CORE: `web/src/app/workspaces/[id]/projects/[projectId]/issues/[issueId]/page.tsx` (+ `loading.tsx` / `not-found.tsx` / `error.tsx`) — the identical real skeleton → fetch-and-guard → render progression as LIVE's own Workspace Detail and TEAMWORK's own Project Detail.

STRETCH: one real line changed in each of the three real dynamic route pages: `res.status === 404` → `!res.ok`.

CHALLENGE: no code — a real, written, self-researched answer.

Files changed

- `web/src/app/workspaces/[id]/projects/[projectId]/issues/[issueId]/page.tsx` (new)
- `web/src/app/workspaces/[id]/projects/[projectId]/issues/[issueId]/loading.tsx` (new)
- `web/src/app/workspaces/[id]/projects/[projectId]/issues/[issueId]/not-found.tsx` (new)
- `web/src/app/workspaces/[id]/projects/[projectId]/issues/[issueId]/error.tsx` (new)
- `web/src/app/workspaces/[id]/page.tsx` (STRETCH: one line)
- `web/src/app/workspaces/[id]/projects/[projectId]/page.tsx` (STRETCH: one line)
- `web/src/app/workspaces/[id]/projects/[projectId]/issues/[issueId]/page.tsx` (STRETCH: one line)

Step-by-step implementation

1. **Skeleton:** an `async` function, `params` typed as `Promise<{ id, projectId, issueId }>`, destructured once, echoing `issueId` back — proves the real route resolves before any real fetch exists.
2. **Fetch + guard:** `apiFetch(` /issues/${issueId} `)`, a real `if (res.status === 404) notFound();` guard, then `await res.json()`.
3. **Render:** title/key/status/description, plus `reporterId` / `assigneeId` shown as real raw ids (never a fabricated name — `GET /users/:id` is genuinely protected until Session 25).
4. **`loading` / `not-found` / `error`:** the identical real file-based trio as the two earlier pages, applied a third time.

5. **The real bug, reproduced on purpose:** `curl http://localhost:3000/issues/abc` returns a real `400 Bad Request` (the real Nest `ParseIntPipe`), never a `404`. Every CORE page's own `=== 404` check lets this real `400` fall straight through, and `res.json()` returns `{message, error, statusCode}` — an object with none of the real fields the page expects, silently rendering `undefined` in their place.
6. **The real fix:** `!res.ok` is true for every real non-2xx status, not only `404` — applied to all three real dynamic route pages, then re-verified against the real happy path too.

Common mistakes

- Fixing the real bug on only the page where it was first noticed — all three real pages share the identical mistake, since all three were built from the identical pattern.
- Checking `res.status >= 400` instead of `!res.ok` — functionally equivalent here, but `.ok` is the real, direct Fetch API property built for exactly this.
- Resolving `assigneeId` / `reporterId` into a fabricated name instead of the real raw id.

Exact testing instructions

BASH

```
# terminal 1
cd api && PORT=3000 npm run start

# terminal 2
cd web && npm run dev

# terminal 3
curl -s http://localhost:3001/workspaces/1/projects/1/issues/1 | grep -o "<h1>.*</h1>"
curl -s http://localhost:3001/workspaces/1/projects/1/issues/999999 | grep -o "No issue
found with that id\."
curl -s http://localhost:3001/workspaces/1/projects/1/issues/abc | grep -o "No issue found
with that id\."
curl -s http://localhost:3001/workspaces/abc | grep -o "No workspace found with that id\."
curl -s http://localhost:3001/workspaces/1/projects/abc | grep -o "No project found with
that id\."
cd web && npx tsc --noEmit && npm run build
```

Every real not-found case shows its own real text; the real existing issue still renders correctly; `tsc` / `build` both exit clean.

What you should understand after studying

Why a real, non-exhaustive status check (`=== 404`) can silently fail in a way that is easy to miss during normal happy-path testing, and why testing the real backend directly (`curl`) before assuming what a frontend page does is a genuinely faster way to find this class of bug than guessing from the rendered UI alone.